data should come from a `SecureRandom` object. In other languages you should read the data from `/dev/urandom` (on Linux) or from an appropriate operating system call such as `getrandom(2)` on Linux or `RtlGenRandom()` on Windows.

> **DEFINITION** In information security, ***entropy*** is a measure of how likely it is that a random variable has a given value. When a variable is said to have 128 bits of entropy, that means that there is a 1 in 2^{128} chance of it having one specific value rather than any other value. The more entropy a variable has, the more difficult it is to guess what value it has. For long-lived values that should be un-guessable by an adversary with access to large amounts of computing power, an entropy of 128 bits is a secure minimum. If your API issues a very large number of tokens with long expiry times, then you should consider a higher entropy of 160 bits or more. For short-lived tokens and an API with rate-limiting on token validation requests, you could reduce the entropy to reduce the token size, but this is rarely worth it.

What if I run out of entropy?

It is a persistent myth that operating systems can somehow run out of entropy if you read too much from the random device. This often leads developers to come up with elaborate and unnecessary workarounds. In the worst cases, these workarounds dramatically reduce the entropy, making token IDs predictable. Generating cryptographically secure random data is a complex topic, and not something you should attempt to do yourself. Once the operating system has gathered around 256-bits of random data, from interrupt timings and other low-level observations of the system, it can happily generate strongly unpredictable data until the heat death of the universe. There are two general exceptions to this rule:

- When the operating system first starts it may not have gathered enough entropy and so values may be temporarily predictable. This is generally only a concern to kernel-level